Serilog; the other two are optional, depending on what you want to configure. They enable you to refer to the `LogEventLevel` and the `JsonFormatter` during configuration, respectively.

Listing 12.17 The using directives required for Serilog

```
using Serilog;                      ❶
using Serilog.Events;               ❷
using Serilog.Formatting.Json;      ❸
```

❶ You will always need this.

❷ Optional—enables you to refer to LogEventLevel during configuration

❸ Optional—enables you to refer to the JsonFormatter during configuration

The next block of code is placed immediately after the `using` directives and before the `WebApplication.CreateBuilder` call.

Listing 12.18 Example Serilog configuration using the fluent API

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Override("Microsoft.AspNetCore", LogEventLevel.Warning)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore", LogEventLevel.Warning)
    .WriteTo.Console()
    .WriteTo.File(path: "Logs\\log.json",
     ➥ formatter: new JsonFormatter(), rollingInterval: RollingInterval.Day)
    .CreateLogger();
Log.Information("Starting up");
```

You use the static `Log.Logger` method to create a logger with the specified configuration. Your configuration specifies a minimum log level of `Warning` for the `Microsoft AspNetCore` and `EntityFramework` categories to reduce log verbosity. You use the `WriteTo` method to configure sinks. Your chosen sinks are the console and the file. You will use the default settings for the console output, but you have configured the file logger to output using JSON. You have also specified that a new file should be created each day via the